

System Audit Compliance Report

Prepared by: [Solomon Mwaura]

Organization: FinSecure IT Security Team

Date: February 12, 2026

Section 1: Objective of the Lab

In this lab, the objective is to address **FinSecure's internal audit policies** by developing a Python automation script. The specific compliance requirements being addressed are:

1. **Data Confidentiality:** Ensuring strict file system permissions on sensitive directories (specifically `/secure_data`).
2. **Access Control:** Monitoring active user sessions to detect anomalies or policy violations regarding concurrent access.
3. **System Availability:** Monitoring disk space utilization to prevent service outages caused by storage exhaustion.

This task is crucial because, as a financial organization, FinSecure handles sensitive monetary and personal data. Manual audits are prone to human error and are often too slow to catch real-time risks. By automating these checks, we ensure continuous compliance with security standards, protect proprietary data from unauthorized access, and ensure the high availability of critical financial services.

Section 2: Explanation of Script Functions

1. Function Name: `check_file_permissions()`

- **What it does:** This function verifies that the directory `/secure_data` is configured with strict access controls, ensuring that only the file owner has access, while "Group" and "Others" are completely restricted.
- **How it works:**
 - It uses the `os.stat()` method to retrieve the status of the specific directory.
 - It utilizes bitwise operators (`&`) to check the file mode bits against `stat.S_IRWXO` (permissions for others) and `stat.S_IRWXG` (permissions for group).

- The logic checks if these bits are set to 0. If they are zero, it means no permissions exist for group/others, and the directory is deemed compliant.
- **Why it is important:** In a Linux environment, permissive settings (like 777 or 755) on financial data directories can allow unauthorized users or compromised accounts to read or modify sensitive records. Enforcing "Owner Only" access is a fundamental principle of **Least Privilege**.

2. Function Name: `check_active_sessions()`

- **What it does:** This function counts the number of users currently logged into the system and alerts administrators if the count exceeds the established threshold of 5 concurrent sessions.
- **How it works:**
 - It uses the `subprocess` module to execute the system shell command `who`, which lists logged-in users.
 - It captures the standard output (`stdout`), splits the output by newlines to create a list, and calculates the length of that list.
 - It compares the length against the integer 5.
- **Why it is important:** An unusually high number of active sessions can indicate a potential security incident, such as a **Brute Force attack** in progress, a botnet infection, or unauthorized sharing of credentials. Limiting concurrent sessions is a key control in identity and access management (IAM).

3. Function Name: `check_disk_space()`

- **What it does:** This function calculates the percentage of disk space currently in use on the root (`/`) partition and warns if usage exceeds 80%.
- **How it works:**
 - It utilizes the `shutil.disk_usage("/")` method to retrieve total, used, and free bytes.
 - It performs a mathematical calculation: $(\text{Used} / \text{Total}) * 100$ to determine the usage percentage.
 - It evaluates this percentage against the threshold (80%).
- **Why it is important:** This directly addresses **Availability**, one of the three pillars of the CIA Triad. If a server runs out of disk space, it cannot write logs (blinding security teams), databases may corrupt, and the OS may crash, leading to financial service downtime.

Section 3: Application of Risk Identification Concepts

Context: The lab scenario aligns with the broader concept of **Continuous Security Monitoring (CSM)**. FinSecure cannot rely on annual audits; they need real-time validation that their security posture matches their written policy.

Process: The script represents the **Risk Assessment and Mitigation** process:

1. **Risk Identification:** We identified three specific risks: Unauthorized data access, unauthorized entry (excessive sessions), and denial of service (disk full).
2. **Risk Analysis:** We determined specific metrics (thresholds) that define a "risky" state (e.g., >80% disk usage, >5 users, or any permission bits for 'others').
3. **Risk Mitigation:** The script provides immediate detection, allowing the admin to remediate the issue before it becomes an incident.

Outcome: The tasks completed in this script mitigate technical risks by providing an automated "early warning system." Instead of waiting for a server to crash or data to be stolen, the script allows FinSecure to act proactively, thereby achieving the goal of **operational resilience**.

Section 4: Testing and Results

To validate the script, I would perform the following tests in a safe, controlled environment:

Test 1: File Permissions

- **Action:** Create a directory `/secure_data` and run `chmod 777 /secure_data`. Run the script.
- **Expected Result:** The script should print: `Permissions for '/secure_data' are NOT compliant.`
- **Action:** Run `chmod 700 /secure_data`. Run the script.
- **Expected Result:** The script should print: `Permissions for '/secure_data' are compliant.`

Test 2: Active Sessions

- **Action:** Open 6 separate terminal windows (creating 6 sessions). Run the script.
- **Expected Result:** The script should print: `Too many active sessions: Compliance NOT met.`

Test 3: Disk Space

- **Action:** Since we cannot easily fill a hard drive for a test, I would temporarily modify the script threshold to `1%` (or a value lower than current usage). Run the script.

- **Expected Result:** The script should print: `Disk space utilization is NOT compliant.`

Compliance Indicator: If the script runs and returns "Compliant" for all three checks, it indicates that the server currently meets FinSecure's baseline security standards. Any "NOT Compliant" output serves as an immediate **Incident Response trigger**.

Section 5: Conclusion

In this lab, I gained hands-on experience creating a Python-based compliance auditing tool for FinSecure. By implementing functions to check file permissions, session counts, and disk usage, I developed a deeper understanding of how abstract policies (like "Protect Confidentiality") translate into concrete code logic (like `stat.S_IRWXO`).

One challenge I encountered was understanding the bitwise operators required to check file permissions programmatically, but referencing the `stat` module documentation clarified how to isolate specific permission bits.

This lab emphasized that automation is essential for modern cybersecurity. The script effectively contained:

1. `check_file_permissions`: To enforce Least Privilege on sensitive data.
2. `check_active_sessions`: To monitor for anomalous user activity.
3. `check_disk_space`: To ensure system availability and prevent data loss.

By deploying this script, FinSecure can significantly reduce the time required for audits and improve their reaction time to potential security misconfigurations.